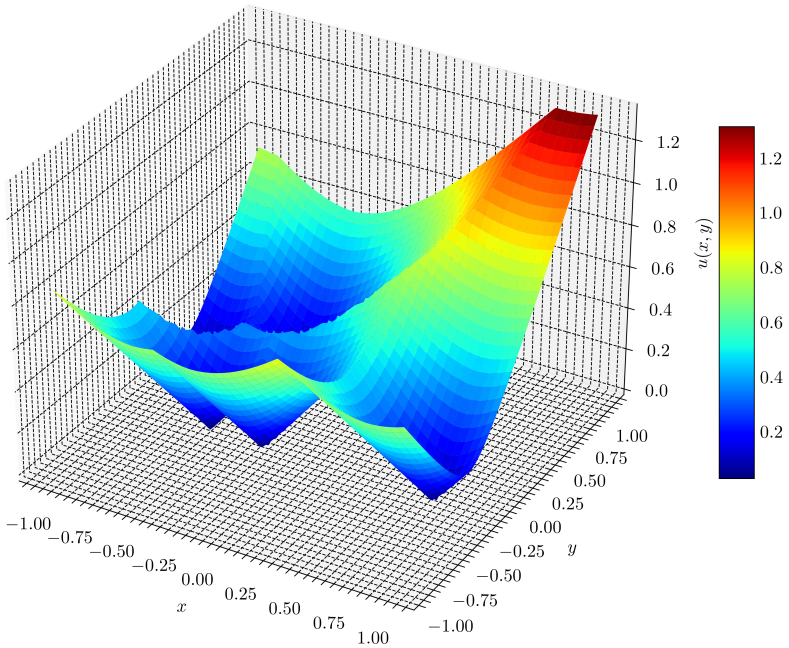
THE FAST-SWEEPING METHOD FOR EIKONAL EQUATIONS & AN APPLICATION TO OPTIMAL CONTROL

Hugo Belzil

MATH 478 - Computational Methods for Applied Mathematics
Instructor: Dr. J.C. Nave
Semester: Winter 2025
McGill University

Numerical Solution on $[-1,1]^2$, with $N = 301$, 5 random points in Γ



July 30, 2026

Contents

1 Introduction **2**

2 Theory **2**
2.1 Origins and derivation . . . 2
2.2 Distance function in 2D . . . 3

3 Numerical methods **3**
3.1 Discretization and scheme . . . 3
3.2 Properties of the Fast-Sweeping method . . . 4
3.3 Implementation details . . . 5

4 Results **6**
4.1 Distance functions in 2D . . . 6
4.1.1 Convergence . . . 7
4.2 Distance functions in 2D with several boundary points . . . 8
4.3 General Eikonal equations . . . 9
4.3.1 Convergence . . . 9

5 An application of Eikonal equations : Path planning **10**
5.1 Bilinear interpolation . . . 10
5.2 Discretization of the ODE system . . . 11
5.3 Example : light refraction with Snell's law . . . 12
5.4 Paths avoiding objects . . . 15

6 Conclusion **16**

1 Introduction

The Eikonal equations have several applications across many fields of mathematics, engineering, and others. They are particularly used for solving optimization problems, as we will see in this project. These equations associated to some side-condition, are of the form

$$\begin{cases} |\nabla u(\vec{x})| = f(\vec{x}), & \vec{x} \in \Omega \\ u(\vec{x}) = \phi(\vec{x}), & \vec{x} \in \Gamma \subset \Omega \end{cases} \tag{1}$$

where $|\cdot|$ denotes the Euclidean norm in $\mathbb{R}^n$, and $u, f, \phi : \mathbb{R}^n \to \mathbb{R}$.

In this report, we will focus on solving Eikonal equations in 2 space dimensions. We will introduce the discretization and fast-sweeping method proposed by H.Zhao in 2008 [1], considering several scenarios for $f(x, y)$ and Γ, and observing some numerical results. Finally, we will use these results and apply them to an optimal control problem : path planning. The Python implementation of the Fast-Sweeping method and all scripts used in this report are available on GitHub at: github.com/hugobelzil/eikonal_project.

2 Theory

2.1 Origins and derivation

Eikonal equations are common equations, encountered in several fields of the sciences. They first arose in geometric optics problems on the propagation of light waves, the term *eikonal* being a germanization of the greek word *eikon*, meaning "image" or "icon"[2]. We will see in section (5.3) an example where it applies to the refraction of light in changing media.

These equations however arise in several other fields, including optimal control theory and robotics. The solutions of eikonal equations can also be used to compute optimal paths in time, as we will see in section (5).

In particular, the differential equation (1) can be derived from level-set equations, for the propagation of a front for example. If we let the 0-level set of a function $\phi(\vec{x}, t)$ represent a curve or a surface moving in space (being a front), we need the function ϕ to satisfy the advection equation:

$$\phi_t + F(\vec{x})|\nabla\phi| = 0 \tag{2}$$

where $F : \vec{x} \to \mathbb{R}$ represents the velocity at which the point $\vec{x}$ on the curve (or surface) moves. If we restrict F to be strictly positive, we force the curve to move "outward" and extend in space. This assumption allows us the make the ansatz that ϕ can be expressed as : $\phi(\vec{x}, t) = \psi(\vec{x}) - t$. The reason for this is the following : this form allows us to think of $\psi(\vec{x})$ as the arrival time of the curve (or front) to the point $\vec{x}$ in space. Indeed, recall that the curve is described as the 0-level set of $\phi(\vec{x}, t)$. We can now plug-in this ansatz into (2) and get:

$$\begin{aligned} &(\psi(\vec{x}) - t)_t + F|\nabla(\psi(\vec{x}) - t)| = 0 \\ \implies &-1 + F|\nabla\psi| = 0 \\ \implies &|\nabla\psi| = \frac{1}{F} \end{aligned}$$

which indeed gives an Eikonal equation. In the front-propagation context, we can hence interpret the solution as the following : the arrival times ψ in space of a front starting at a point γ and propagating in a medium with speed F must solve the eikonal equation

$$|\nabla\psi(\vec{x})| = \frac{1}{F(\vec{x})}, \quad \psi(\gamma) = 0$$

When $F \equiv 1$, the front is moving through a uniform media, at the same speed everywhere. Hence, we can view ψ as the distance to γ (see section 2.2). In the analysis above, we let γ be a single point in space, but the front could also originate from a curve, or several points in space, as we will see later in this report.

2.2 Distance function in 2D

An important application of Eikonal functions is the computation of Euclidean distances in $\mathbb{R}^n$. In particular, let us consider the following problem : suppose that we are given a finite set of points $\Gamma = \{\gamma^{(1)}, \dots, \gamma^{(m)}\}$, and for each $\mathbf{x} = (x, y)$ in our computational domain, we would like to compute the shortest distance to Γ. Hence, we would like to have a function $d(x, y)$ such that

$$d(\mathbf{x}) = \min_{\gamma \in \Gamma} \|\mathbf{x} - \gamma\|_2 \tag{3}$$

Consider this problem in $\mathbb{R}^2$, if we let (x, y) such that its closest point in Γ is some point $\gamma^* = (\gamma_1, \gamma_2)$, which is the unique minimizer of (3), then:

$$d(x, y) = \sqrt{(x - \gamma_1)^2 + (y - \gamma_2)^2}$$

$$\implies \frac{\partial d}{\partial x} = \frac{x - \gamma_1}{\sqrt{(x - \gamma_1)^2 + (y - \gamma_2)^2}}, \quad \frac{\partial d}{\partial y} = \frac{y - \gamma_2}{\sqrt{(x - \gamma_1)^2 + (y - \gamma_2)^2}}$$

$$\implies |\nabla d(x, y)| = \sqrt{\left(\frac{\partial d}{\partial x}\right)^2 + \left(\frac{\partial d}{\partial y}\right)^2} = 1$$

Indeed, $d(x, y)$ solves the Eikonal equation where $f(x, y) \equiv 1$, and naturally we have $d(x, y) = 0$ where $(x, y) \in \Gamma$ (the distance to the boundary is 0 when the point lies on the boundary).

Note that we made the important assumption that γ^* was the only minimizer of (3). However, if card$|\Gamma| \geq 2$, then there exist points in $\mathbb{R}^2$ for which (3) does not admit a single minimizer, but rather infinitely many. In this case, $d(x, y)$ becomes non-differentiable at these points, which will be discussed in our analysis.

3 Numerical methods

3.1 Discretization and scheme

H. Zhao proposed the fast-sweeping method in [1] to solve Eikonal equations numerically. He proposed the following discretization, which uses Gauss-Seidel iterations. In 2 dimensions, given a space step h, the scheme is:

$$((u_{i,j} - u_{xmin})^+)^2 + ((u_{i,j} - u_{ymin})^+)^2 = f_{i,j}^2 h^2$$

where

- $(x)^+ = \max(x, 0)$
- $u_{i,j} \approx u(ih, jh)$, $u_{xmin} = \min\{u_{i-1,j}, u_{i+1,j}\}$ and $u_{ymin} = \min\{u_{i,j-1}, u_{i,j+1}\}$

Figure (1) shows the dependence of u_{ij} on its neighbouring points during an update. The updates of the points $u_{i,j}$ must be done by "sweeping" the domain in all 4 possible directions (or 2^d directions in $\mathbb{R}^d$), after having initialized a grid. Although the order of the sweeps does not matter, the sweeps in 2 dimensions can be done as follows, assuming a square domain:

- **Sweep 1 :** Start from the bottom-left corner of the domain, updating the points from left to right. Hence, the last point updated is the top-right corner one.

- **Sweep 2 :** Start from the bottom-right corner of the domain, updating the points from right to left. This gets us to the last point updated being the top-left corner one.
- **Sweep 3 :** Start from the top-left corner of the domain, updating from left to right. Similarly, the last point to be updated is the bottom-right corner one.
- **Sweep 4 :** Last possible direction. Start from top-right corner of the domain, updating the points from right to left. The last point to be updated is the bottom left corner.

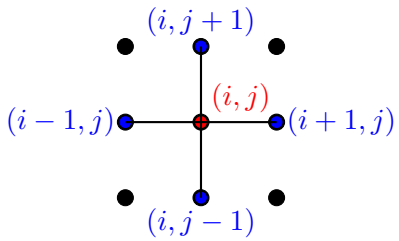


Figure 1: Discretization for the Fast-Sweeping method in 2D

Hence, a full iteration of the scheme can be described as follows:

- **(1)** Initialize the computational domain by setting all points to a large value M (this value should be larger than the largest value of the solution u in the domain).
- **(2)** Place the boundary conditions $\Gamma = \{\gamma_1, \dots, \gamma_m\}$ on the domain. These points are fixed and should never be updated!
- **(3)** Apply a batch of sweeps to the domain, i.e. sweeps 1 to 4. If needed, sweep again the domain until convergence is met. To monitor convergence, compute at the k-th sweep $E = \max_{i,j} \left| u_{i,j}^{(k)} - u_{i,j}^{(k-1)} \right|$ where $u_{i,j}^{(k)}$ denotes the solution on the stencil (i, j) after the k-th sweep.

During step (**3**) in the above, one needs to solve the following equation for every point $u_{i,j}$:

$$[(u_{i,j} - a)^+]^2 + [(u_{i,j} - b)^+]^2 = f_{i,j}^2 h^2 \tag{4}$$

where $a = \min\{u_{i-1,j}, u_{i+1,j}\}$ and $b = \min\{u_{i,j-1}, u_{i,j+1}\}$. The solution to (4) is given by:

$$\bar{u} = \begin{cases} \min\{a, b\} + f_{i,j} h & \text{if } |a - b| \geq f_{i,j} \\ \frac{a+b+\sqrt{2f_{i,j}^2 h^2 - (a-b)^2}}{2} & \text{if } |a - b| < f_{i,j} \end{cases}$$

Finally, $u_{ij} \leftarrow \min\{u_{ij}, \bar{u}\}$.

3.2 Properties of the Fast-Sweeping method

We report in this section several properties of the fast-sweeping method.

Proposition 3.1 (**Monotonicity of the scheme**). *The fast-sweeping scheme in decreasing in the sweeps :* $\forall\, i, j : u_{ij}^{(k)} - u_{ij}^{(k-1)} \leq 0$

Proposition 3.2 (**Number of sweeps and accuracy for distance functions***). *For the distance function in* $\mathbb{R}^n$ *(*$F(x, y) \equiv 1$*), the fast-sweeping method converges in* 2^n *sweeps. However, this needs to be nuanced and argued differently.*

If $|\Gamma| = 1$*, the scheme converges exactly after* 2^n *sweeps. Hence, in 2 dimensions, a* 5^{th} *sweep will not update the numerical solution* u^h*. In this case,* $\|u - u^h\|_\infty = \mathcal{O}(|h \log h|)$

If $|\Gamma| > 1$*, some points in the computational domain will be at the same distance to at least 2 boundary points*

in Γ. This creates an interaction between these boundary points and results in the following. After 2^n sweeps, the scheme is still $\mathcal{O}(|h \log h|)$, but further sweeps will still increase the accuracy of the numerical solution (i.e. sweeps still update the numerical solution). In this case, after $2^n + 1$ sweeps or more, $\|u - u^h\|_\infty = \mathcal{O}(h)$.
***REMARK :** this proposition is our understanding from the paper of H. Zhao, but needs to be verified. This will be updated in a future version of this report.*

Proposition 3.3 (*Number of sweeps and accuracy for general equations). *In the case where the velocity field $F(x, y)$ is not constant, the number of sweeps needed required to meet convergence depends on the characteristic curves, (and thus, on F).*

If $|\Gamma| = 1$ and F is smooth, this is a special case where we can have an accuracy of order $h \log h$, as for the distance function case above. This is observed non-rigorously later in this report.

If Γ and F are such that the solution u develops non-differentiabilities, it appears that the scheme's accuracy becomes of order $\sqrt{h}$.
***REMARK :** As for the proposition above, this needs to be verified and will be updated in the future.*

3.3 Implementation details

We implement the fast-sweeping method in Python. All scripts are available in the GitHub repository (link the introduction).
We replicate domains with the help of a class named `computational_domain` , equipped with the method `ComputationalDomain(N,a,b,c,d)` which instantiates an object with several attributes, including the attribute `grid` which is an $N \times N$ Numpy 2D array [3], representing the square $[a, b] \times [c, d] \subset \mathbb{R}^2$. The constructor of the class sets the initial value M mentioned above to 100, but can be chosen by the user.
This class is also equipped with the method `Gamma(coordinates)` which sets the boundary conditions in the domain. `coordinates` must be a list of 2D coordinates given as a indices, for example `coordinates = [(0,0), (1,0), (4,4)]`. Note that this method assumes that the coordinates are given in Python indexing. For example, the code below would help create the following domain:

```
from computational_domain import ComputationalDomain
domain = ComputationalDomain(N = 6, a = -1, b = 1, c = -1, d = 1)
domain.Gamma([(0,0), (2,1), (4,3), (1,4)])
print(domain.grid)
```

Listing 1: Instance of computational domain

$$\begin{bmatrix} 0 & 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 0 & 100 \\ 100 & 0 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 0 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 & 100 \end{bmatrix}$$

The code in the repository comes with 2 classes, `eikonal_distance_2D` and `eikonal_general_F`, which help treating distinctly the cases where $F(x, y) \equiv 1$ and $F(x, y) \not\equiv 1$. Both classes come with a method named `EikonalSolver()`, both taking as input an object of the class `computational_domain` described above. Additionally, the `EikonalSolver` method from the general function F takes as input the function F. An example for both cases is available below.

```
from computational_domain import ComputationalDomain
from solvers.eikonal_distance_2D import EikonalSolver
domain = ComputationalDomain(N = 501, a = -1, b = 1, c = -1, d = 1)
domain.Gamma([(250, 250)])

solver = EikonalSolver(domain = dom)
```

```
solver.BatchSweeps(k = 1)
print(solver.grids_after_sweeps[-1])
```

Listing 2: Instance of an Eikonal Solver for the 2D distance function

In this code above, we solve the Eikonal equation for the distance to the origin over the square $[-1,1]^2$ with $N = 501$ grid points. The `solver` object stores the grid after each sweep, and line 8 allows to access the last grid (after the fourth sweep), which is stored as a Numpy array. The snippet below extends the method to the general function F.

```
from computational_domain import ComputationalDomain
from solvers.eikonal_general_F_2D import EikonalSolver
domain = ComputationalDomain(N = 501, a = -1, b = 1, c = -1, d = 1)
domain.Gamma([(250, 250)])
eps = 1e-6

def F(x,y):
    return 1 + x**2 + y**2

solver = EikonalSolver(domain = dom, F = F)
solver.SweepUntilConvergence(epsilon = eps, verbose = True)
print(solver.grids_after_sweeps[-1])
```

Listing 3: Instance of an Eikonal Solver for a general velocity F

In this code above, our aim is to solve the Eikonal equation in the same domain, except that $F(x,y) = 1 + x^2 + y^2$. Observe that we now use the method `SweepUntilConvergence`, which allows us to monitor the convergence after each sweep, and thus avoid unnecessary ones. After each sweep, the method computes the maximum change ($\|\cdot\|_\infty$) between the new grid and the one that had resulted from the previous sweep. If this change is below some threshold `epsilon`, the method stops.

4 Results

4.1 Distance functions in 2D

As argued above in section 2.2, the Eikonal equation can be used to compute the shortest distance to a boundary $\Gamma \subset \mathbb{R}^n$. Therefore, we test our implementation for the BVP:

$$\begin{cases} \|\nabla u(\vec{x})\|_2 = 1, & \vec{x} \in [-1,1]^2 \\ u(0,0) = 0 \end{cases} \tag{5}$$

The solution to this BVP is the 2D cone, given by $u(x,y) = \sqrt{x^2+y^2}$. Therefore, we can easily compare our numerical solution to the analytic one, and study its convergence as well. For solving numerically (5), we discretize the square $[-1,1]^2$ uniformly with $N = 257 \implies h \approx 0.0078$. The numerical solution converges after exactly 4 sweeps, as outlined by proposition (3.2).

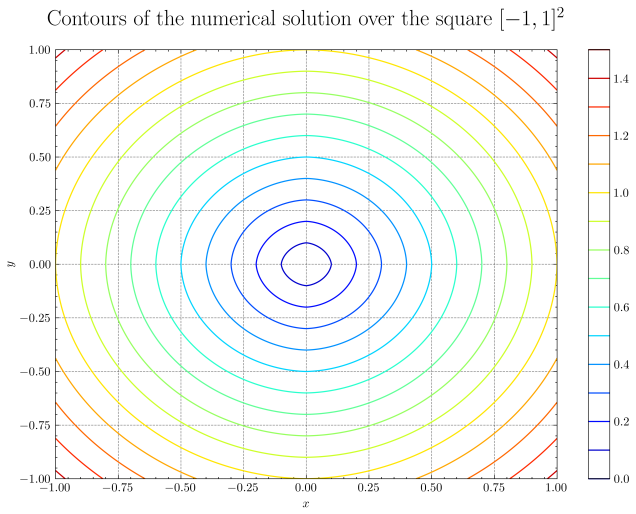


Figure 2: Contour plot of the numerical solution to (5).

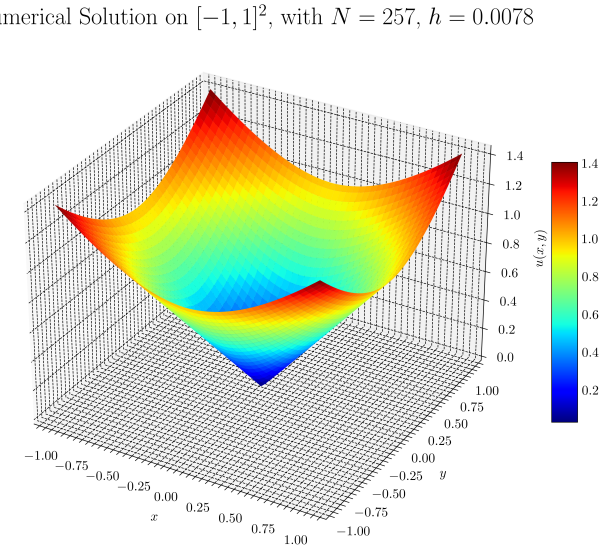


Figure 3: 3D surface of the numerical solution.

Figure (2) shows the contour plots of the numerical solution. Note that since the analytical solution is $u(x, y) = \sqrt{x^2 + y^2}$, we would expect in theory to have perfectly round circles. However, this remains a numerical approximation, explaining why we are seeing "oval" forms rather than perfect circles. If one were to decrease the step size h (i.e. increase N), these ovals would eventually look more and more like perfectly round circles. Figure (3) confirms as well that the numerical solution is somewhat close the cone we would expect.

4.1.1 Convergence

Finally, we can analyze the convergence of the fast sweeping method for the BVP (5). The plot below was obtained was solving numerically the BVP, using $N \in \{201, 221, 241, \ldots, 701\}$, converting to step sizes $h = \frac{N-1}{b-a}$. For every of these values, the numerical solutions is compared to the analytic solution. Specifically, the error is computed with the $\|\cdot\|_\infty$ norm over the square $[-1, 1]^2$.
On figure (4) below, we see that the accuracy of the fast sweeping method indeed seems to be of order $O(|h \log h|)$ in this case.

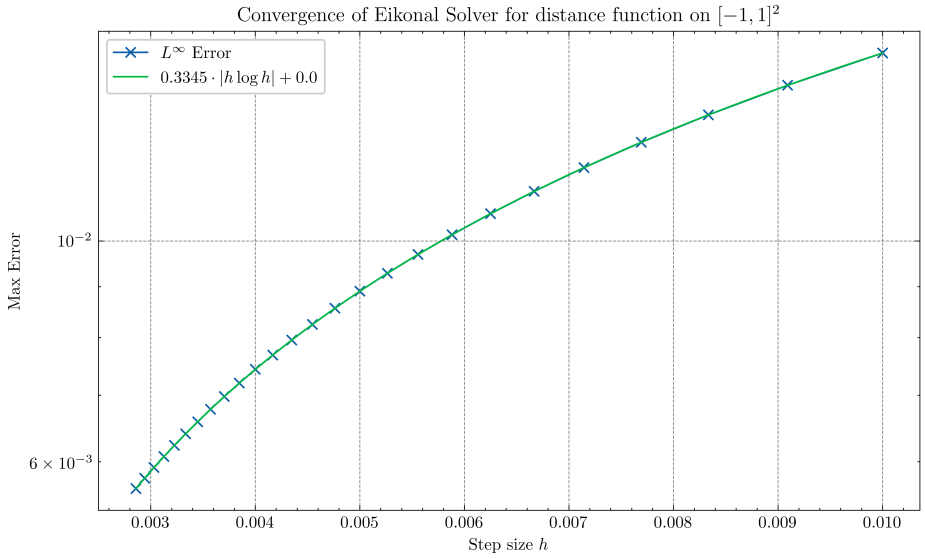


Figure 4: Accuracy of the fast-sweeping method for the distance function to the origin

4.2 Distance functions in 2D with several boundary points

The example in the previous subsection was only considering one boundary value at the origin of the domain, i.e. the set Γ was simply $\Gamma = \{(0,0)\}$. We can introduce several other boundary points, and the solution u picked up by the fast sweeping-method will give at any point (x, y) the distance to the closest point $\gamma \in \Gamma$, as we showed in section (2.2). Indeed consider the case below, where we chose 5 points at random on $[-1,1]^2$, and set these points to be 0 in the domain. Our Eikonal solver then converges to the numerical solution $u(x, y)$ whose contours and 3D surface are given by figures (5) and (6) respectively.
It is easy to see on both the contour plots and on the 3D surface that the solution is non-differentiable on some lines inside the domain. In particular, the points on these lines are the ones lying at the same distance to at least 2 boundary points in Γ. In other words, the problem (3) does not have a unique minimizer for these sets of points.

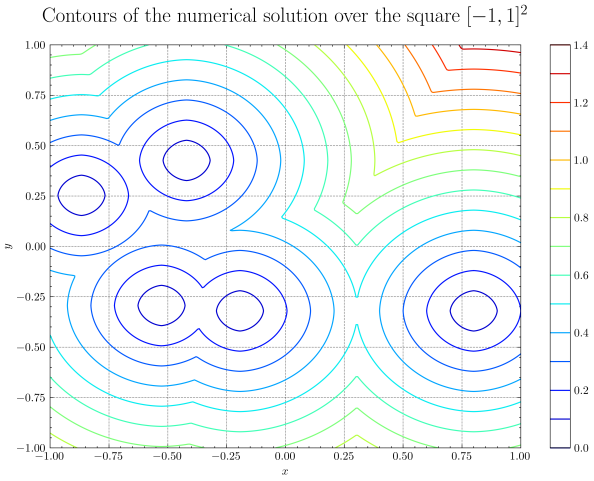


Figure 5: Contours of numerical solution ($|\Gamma| = 5$)

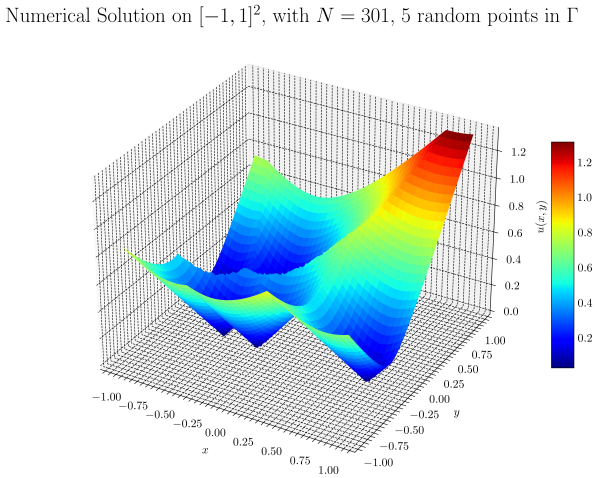


Figure 6: 3D surface of the solution ($|\Gamma| = 5$)

Finally, we look at another case where we introduce several more boundary points in Γ. In the example below, we sampled uniformly at random 30 points in $[-1,1]^2$ and set these points to have value 0. Figure (7) represents a heatmap of the solution. The regions where the solution is non-differentiable are easily seen.

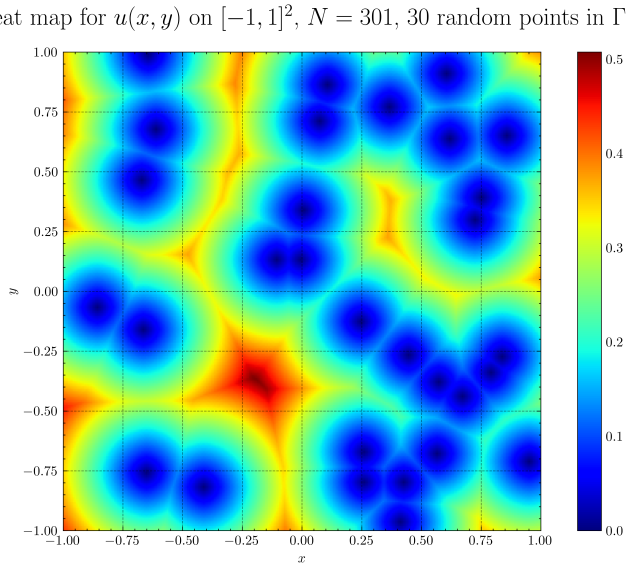


Figure 7: Distance function in 2D for $|\Gamma| = 30$ (random boundary points)

4.3 General Eikonal equations

In this subsection, we now are interested at problems of the form of (1) where $f(x,y) \not\equiv 1$ on the domain. Conveniently, we will also rewrite $f(x,y) = \frac{1}{F(x,y)}$, where $F(x,y)$ can now be thought of as a velocity over the domain, as argued in the derivation in the wave (or front) propagation context. As a result, the solution $u(x,y)$ does not give the distance to the boundary anymore, but rather the "cost" of the travel from (x,y) to Γ. We test our solver for the problem.

$$\begin{cases} |\nabla u(x,y)| = \frac{1}{1+0.5(x^2+y^2)}, & (x,y) \in [-2,2]^2 \\ u(0,0) = 0 \end{cases} \tag{6}$$

One can check that $u(x,y) = \sqrt{2}\arctan\left[\sqrt{\frac{x^2+y^2}{2}}\right]$ solves (6). We use this reference to test our solver with the following parameters : $N = 801$ on the discretized domain $[-2,2]^2$ which gives the step size $h = 0.005$, with convergence criterion $\epsilon = 10^{-6}$. The contours and surface of the numerical solution are displayed below on figures (8) and (9).

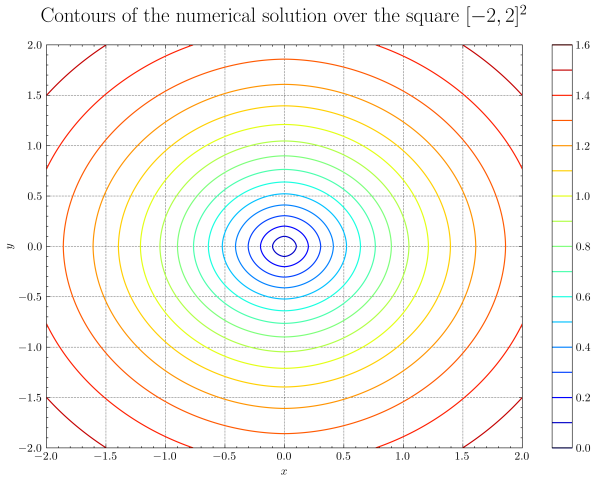


Figure 8: Contours of numerical solution where $F(x,y) = 1 + 0.5(x^2+y^2)$

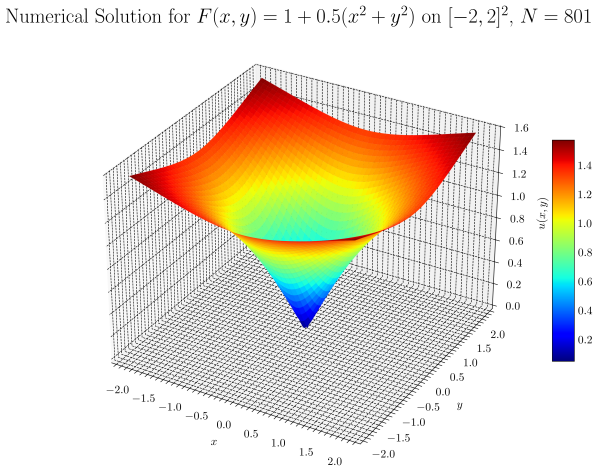


Figure 9: 3D surface of the numerical solution

4.3.1 Convergence

From Zhao, we know that for the general case $1/F(x,y) \not\equiv 1$, we believe that fast-sweeping method also converges to the true solution in $\mathcal{O}(h \log h)$ in this specific case. For the reference problem defined in the section above, with $N \in \{751, 1001, \ldots, 3751\} \implies h \in \{0.0013, 0.001, \ldots, 0.00027\}$, we get the following convergence to the true solution. To reduce the computational cost and increase the accuracy, we reduced the domain to $\Omega = [-0.5, 0.5]^2$. This convergence plot took about 40 minutes to be obtained.
An important point is that we were at first not entirely sure that the order of accuracy would be $h \log h$ for this specific case. In particular, we were hesitating between $\sqrt{h}$ and $h \log h$. This led us to choosing quite small values for h in order to increase our chance of seeing the asymptotic results to kick in.
We fitted two linear models : $\beta_0 + \beta_1\sqrt{h}$ and $\beta_0 + \beta_1 h \log h$ and the R^2 scores for the models were respectively 0.9951 and 0.9999 reinforcing our belief that the accuracy should be of order $h \log h$.

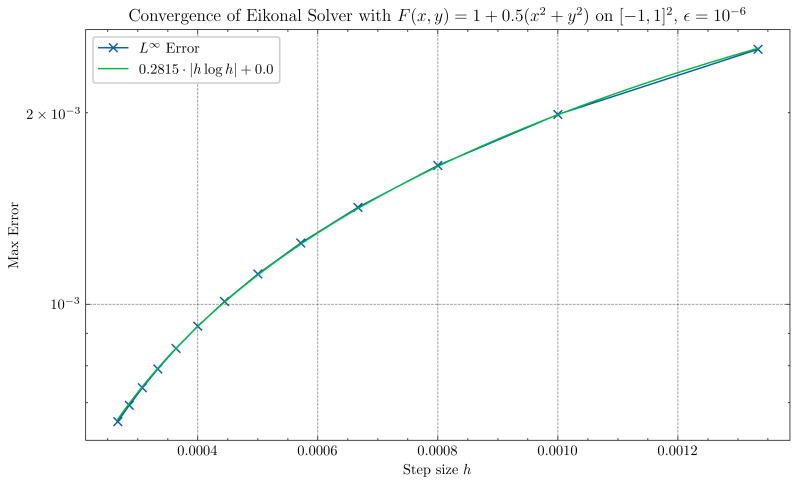


Figure 10: Convergence plot

5 An application of Eikonal equations : Path planning

In this section, we make use of the fast-sweeping method discussed to bridge our solutions with an application : path planning. Our goal is to compute an optimal path over a domain $\Omega \subset \mathbb{R}^2$, starting at a point $X_0 \in \mathbb{R}^2$ to a target set Γ.
In particular, one can show in optimal control theory that given the Eikonal equation set-up that we described above where $F(x, y)$ is the positive function defining the "velocity" inside, say a medium, then the shortest path in time $\mathbf{X}(t) \subset \mathbb{R}^2$ that goes from X_0 to Γ must satisfy the following diffential equation:

$$\frac{d\mathbf{X}(t)}{dt} = -\nabla u(\mathbf{X}(t)), \quad \mathbf{X}(0) = X_0 \tag{7}$$

where u is the solution to the Eikonal equation (1). In 2D, equation (7) can be rewritten as :

$$\begin{bmatrix} \dot{x_1}(t) \\ \dot{x_2}(t) \end{bmatrix} = \begin{bmatrix} -\partial_x u(x_1(t), x_2(t)) \\ -\partial_y u(x_1(t), x_2(t)) \end{bmatrix} \tag{8}$$

Several points need to be taken care of. First, observe that (8) is a time-continuous problem, while the solution u obtained with the fast-sweeping method is discretized over a mesh with stepsize h. Therefore, we will need to be able to interpolate ∇u inside the cells of a mesh grid. For this, we will use bilinear interpolation. Secondly, we will have to discretize (8) in order to solve the problem numerically.

5.1 Bilinear interpolation

Since our numerical solutions u to Eikonal equations are available over a finite grid after using the Fast-Sweeping method, we need to be able to interpolate the values of the gradient of u for points that do not lie in the grid, i.e. points that lie inside the cells of the grid. For this, we use bilinear interpolation. A visual explanation is displayed below.

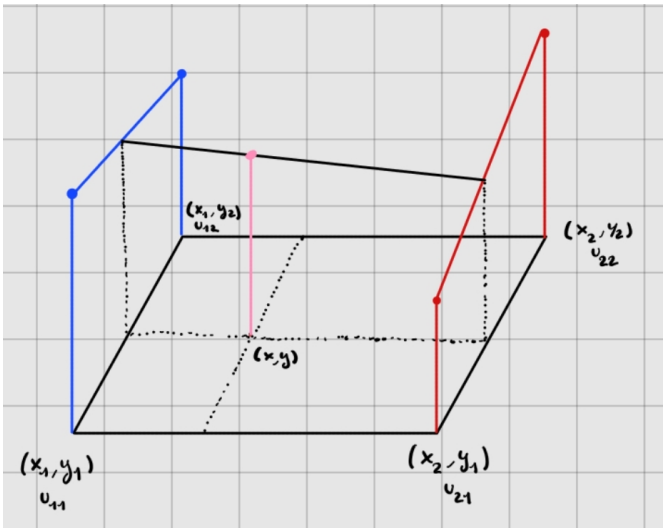


Figure 11: Visual explanation of bilinear interpolation

Given that we know the values u_{11}, u_{12}, u_{21} and u_{22}, we can interpolate $u(x, y)$ inside the cell as follows.
The value of u at the pink point on the graph above can be seen as a convex combination of the value of u on the blue line ($x = x_1$) and the value of u on the red line ($x = x_2$). Hence, our first step is to obtain these 2 values. Let:

$$\alpha = \alpha(y) = \frac{y - y_1}{y_2 - y_1}$$

The interpolated value u_b of u on the blue line and the interpolated value u_r of u on the red line are respectively given by:

$$u_b = \alpha u_{11} + (1 - \alpha)u_{12}, \quad u_r = \alpha u_{21} + (1 - \alpha)u_{22}$$

From this point, we now let

$$\beta = \beta(x) = \frac{x - x_1}{x_2 - x_1}$$

and we can now compute the interpolated value at $\tilde{u}(x, y)$ as a convex combination of u_b and u_r:

$$\begin{aligned} \tilde{u}(x, y) &= \beta u_b + (1 - \beta)u_r \\ &= \beta(\alpha u_{11} + (1 - \alpha)u_{12}) + (1 - \beta)(\alpha u_{21} + (1 - \alpha)u_{22}) \end{aligned} \tag{9}$$

where the parameters α and β are functions of y and x respectively. Therefore, we can take the derivatives of (9) w.r.t. x and y, which is how we obtain the interpolated gradient of u.

5.2 Discretization of the ODE system

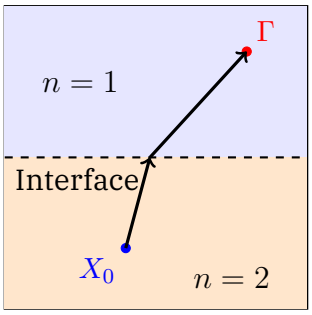
As discussed above, we approach the differential equation (8) using a forward-in-time Euler discretization.Let $x_i^n = x_i(n\Delta t), i = 1, 2$. The scheme is:

$$\begin{cases} \frac{x_1^{n+1} - x_1^n}{\Delta t} = -\tilde{u_x}(x_1^n, x_2^n) \\ \frac{x_2^{n+1} - x_2^n}{\Delta t} = -\tilde{u_y}(x_1^n, x_2^n) \end{cases} \implies \begin{cases} x_1^{n+1} = x_1^n - \Delta t \tilde{u_x}(x_1^n, x_2^n) \\ x_2^{n+1} = x_2^n - \Delta t \tilde{u_y}(x_1^n, x_2^n) \end{cases} \tag{10}$$

which we recognize as being a gradient descent, stopping when $[x_1^n, x_2^n]^T \in \Gamma$, and where $\tilde{u_x}$ and $\tilde{u_y}$ are the interpolated first derivatives of the numerical solution u.

5.3 Example : light refraction with Snell's law

This example covers the refraction of light when it crosses 2 media with different refraction indices. Indeed, some of the first appearances of the Eikonal equations appeared in this type of problems. Consider the following problem : we look at rays of lights in a square domain, where the lower half of the domain's refraction index is 2, and the index is 1 in the upper half plane. Suppose that light is emitted at the point X_0 and we want to know what is the path that the rays emitted at X_0 will take to get to Γ. By Fermat's principle, this path is unique and is the shortest in time. However, the change of medium will make the rays of light "change" direction at the interface.



The refractive index $n = n(x, y)$ is inversely proportional to the speed in the medium. Hence, assume that we have solved the solution to (1) over $[-1, 1]^2$, with $F(x, y) = F_2 = 2$ in the lower half of $[-1, 1]^2$, and $F(x, y) = F_1 = 1$ in the upper half. We know that in each medium, the path will consist of 2 straight lines that will join each other at the interface, on the line $y = 0$. Therefore, one can compute the coordinates of the point x which minimizes the time travel of light over the path from $X_0 = (x_0, y_0)$ and the target $\Gamma = (x_T, y_T)$, i.e.:

$$\min_{x\in[-1,1]} T(x) = \underbrace{\frac{\sqrt{(x-x_0)^2+y_0^2}}{F_1}}_{\text{Time of travel in lower half}} + \underbrace{\frac{\sqrt{(x-x_T)^2+y_T^2}}{F_2}}_{\text{Time of travel in upper half}} \tag{11}$$

Note that we strategically placed the interface on the line $y = 0$, in order to simplify a bit this optimization problem. In our implementation, we chose $X_0 = (x_0, y_0) = (0, -0.5)$ and $\Gamma = (0.5, 0.5)$ which also simplifies the optimization problem. But we are still lazy and use `Scipy` in order to compute the solution to (11).
Our next step is to solve (8), and compare our path to the true path which we obtained above by computing the x-coordinate of the intersection on the interface. We use the `solvers.ODE_backtracer` method available in the repository and obtain the path below.

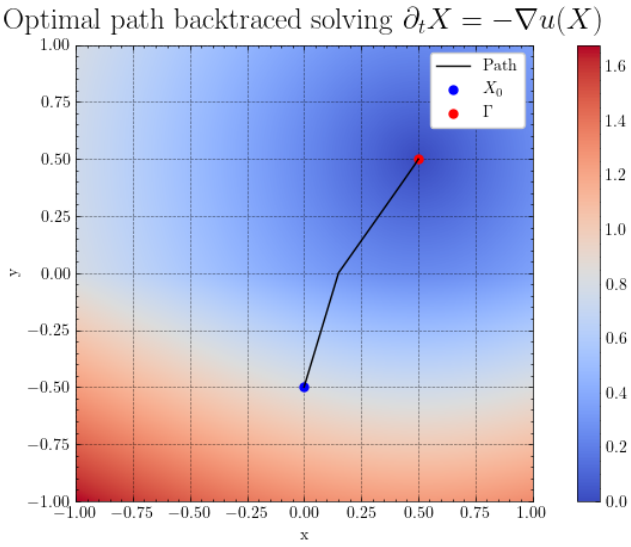


Figure 12: Backtraced path from X_0 to Γ using the numerical solution to the ODE system (8)

This path seems correct but we would like to confirm that our implementation does converge to the true solution. Luckily, the previous work we did to compute the x-coordinate on the line $y = 0$ can be compared with our path. For this purpose, we introduce the Hausdorff distance:

Hausdorff distance between 2 curves : Given 2 curves $S_1, S_2 \subset \mathbb{R}^n$, the Hausdorff distance is a measure that quantifies how close the curves S_1 and S_2 are to each other in space. It is formally defined as:

$$d_H(S_1, S_2) = \max\{\sup_{s_1 \in S_1} d(s_1, S_2), \sup_{s_2 \in S_2} d(S_1, s_2)\}$$

where $d(x, y)$ is a metric on $\mathbb{R}^2$. In our case, we consider the the Hausdorff distance using the Euclidean metric d. It holds that $d_H(S_1, S_2) = 0 \iff S_1 = S_2$, and $S_n \xrightarrow{n\to\infty} S \iff \lim_{n\to\infty} d_H(S_n, S) = 0$. Therefore, after obtaining our true path using Snell's law, we solve the ODE problem with several values of Δt, and compute $d_H(S_{\Delta_t}, S)$ where S_{Δ_t} denotes the numerical path computed, and S denotes the true path computed with Snell's law. We expect $d_H(S_{\Delta_t}, S) \xrightarrow{\Delta_t \to 0} 0$, and that is what we obtain indeed for $\Delta_t \in [1.75h, 3h]$

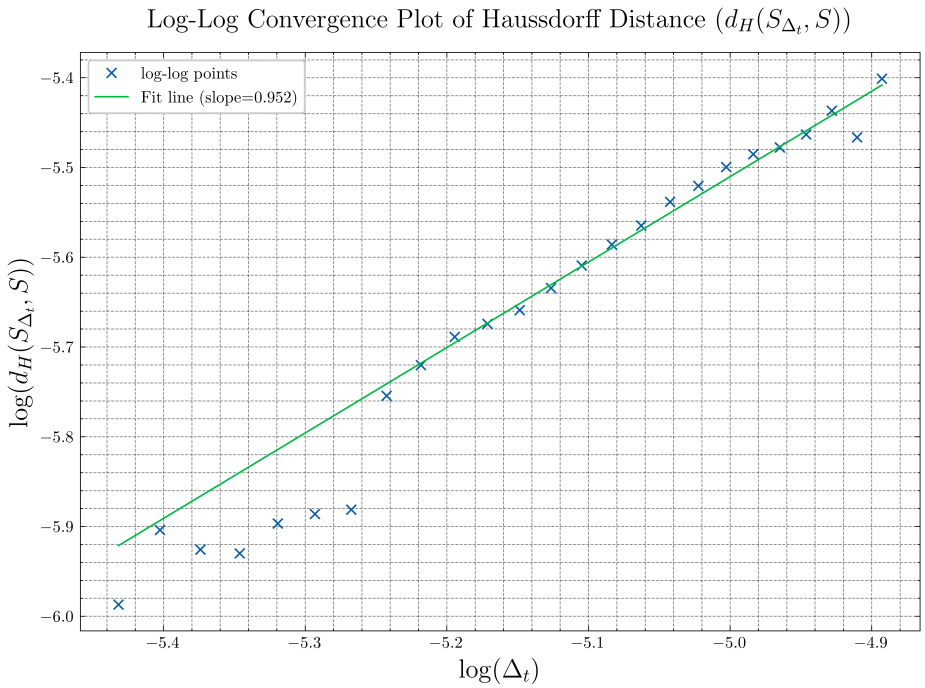


Figure 13: Log-Log plot of the Hausdorff distance versus Δt

The green line was fitted on the 18 first points starting from the right. It had a slope roughly equal to 1, which was expected since the forward-in-time Euler discretization is of order $\mathcal{O}(\Delta_t)$. However, we notice that around $\log(\Delta_t) \approx -5.25$, there seems to be a drop in the log Hausdorff distance, and the errors seems to plateau beyond this point. We verify this assumption by plotting the error over a larger range ($\Delta_t \in [h, 3h]$).

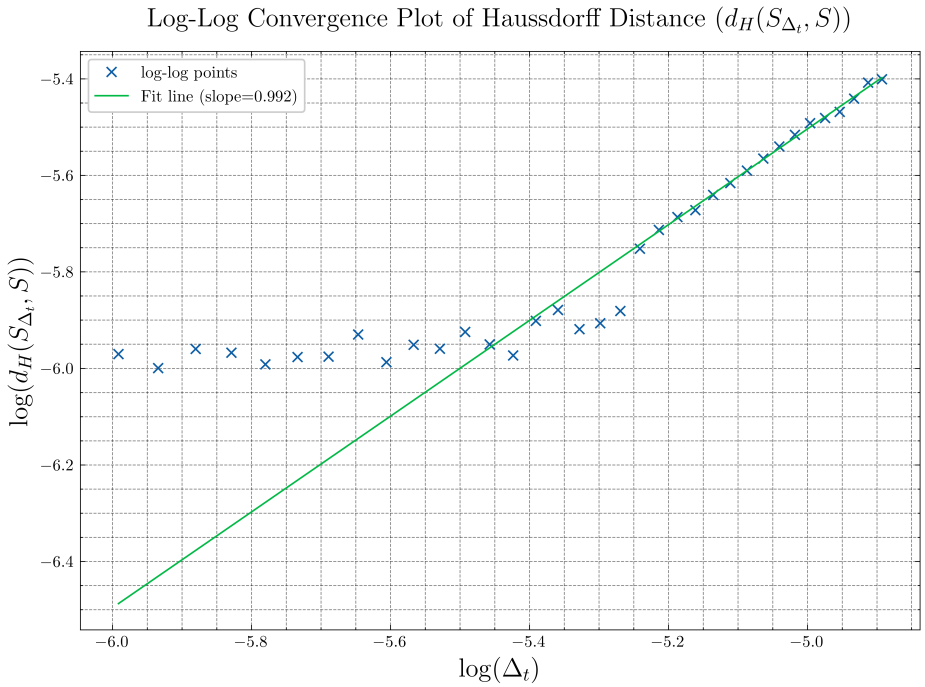


Figure 14: Log-Log plot of the Hausdorff distance versus Δt on a larger window for Δ_t

Our claim is the following : even if our numerical ODE solver's accuracy is of order $\mathcal{O}(\Delta_t)$, this plateau probably occurs at a point, where it is now our bilinear interpolation which causes this error to plateau with respect to Δ_t. In other words, after this , the error induced by lowering Δ_t is lower than the error induced by the bilinear interpolation.

5.4 Paths avoiding objects

In this final section, we use our path planning technique developped in the sections above on some more interesting examples. In the examples below, we let the velocity function $F(x, y)$ take different values on the different regions of the domain, set a starting point and a target and let our ODE solver compute the optimal path. These could be thought of as objects in a room, and the path would be a path computed by a robot, to go from its starting point X_0 to Γ

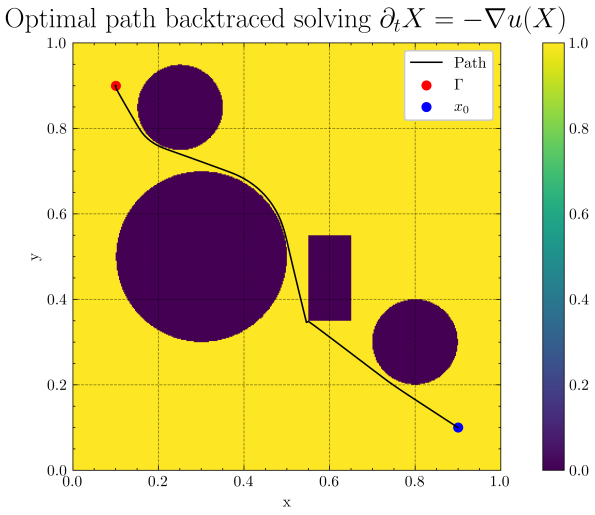


Figure 15: Path backtraced on the domain, the heatmap represents the values of $F(x, y)$

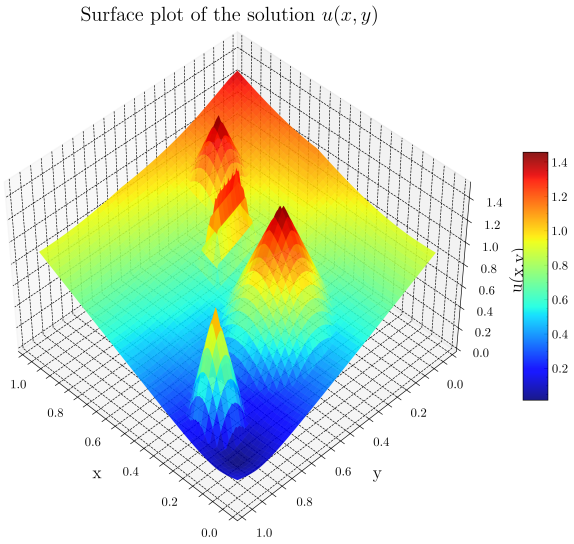


Figure 16: The 3D surface of the solution

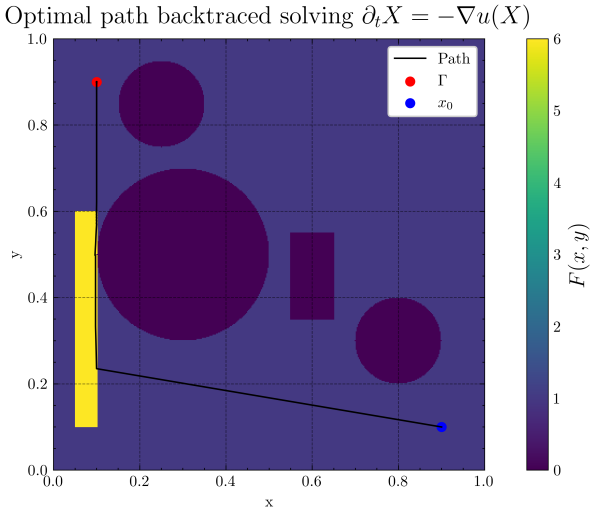


Figure 17: Path backtraced on the domain, the heatmap represent the values of $F(x, y)$. Unlike above, we added a fast lane (yellow)

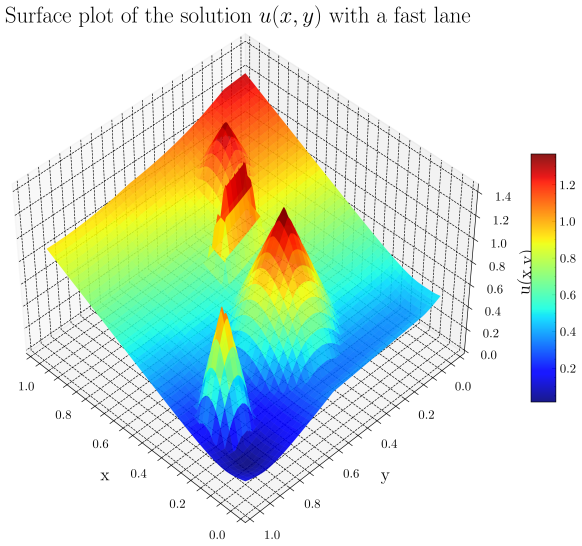


Figure 18: The 3D surface of the solution

It is particularly interesting to see that the path indeed avoids these regions with low velocities, which can be visualized more easily on figure (16). We introduced a ”fast lane” in the second example, where we set $F(x, y) = 6$, and this completely modifies the trajectory of the optimal path in time. Figures (16) and (18) are interesting to compare : adding the fast lane on the left side of the domain significantly lowers the value of u in this region, i.e. the cost it takes to go from there to the target Γ. Thanks to Madelyn Schaffer for her idea of introducing a fast-lane, and allowing me to adapt it to my example!

6 Conclusion

In this report, we aimed at implementing the Fast-Sweeping method for solving Eikonal equations. This Hamilton-Jacobi scheme can easily be extended to high-dimensional problems, and is not computationally too expensive. This method possesses several interesting properties, especially regarding its order of accuracy which heavily depends on the set-up the boundary-value problem. Several points could be addressed in further investigation of the fast-sweeping method. In particular, a few ideas we would like to investigate (and potentially update this report with) are :

- Formalizing the orders of convergence of the method, especially when the velocity field F is not constant or when $|\Gamma| > 1$, mentioned in (3.3). We would also want to look at the regime change of the order of accuracy when computing the distance functions with $|\Gamma| > 1$.
- The accuracy of the Hausdorff distance in section (5.3) eventually reaches a plateau. We claim that higher-accuracy interpolation methods such as bicubic interpolation would help pushing this plateau to happen at lower values of Δ_t, and thus help achieve better accuracies for the path computations. Similarly, we could use some other discretizations the ODE system, and obtain a better accuracy.
- We would also like to go over our Python implementation in order to optimize the algorithm and potentially decrease the runtime. This has been an iterative process since the beginning of this project, but there surely are opportunities left to lower the runtime.
- Finally, we would like to find some topographical data in order to apply our path planning to a real-life scenario. If we can get access to the elevation map of a specific region, we could compute the shortest path in time given these elevations. For example, the gradient of the terrain could be used to inform the velocity field F (i.e. penalize the velocity when the norm of the gradient of the elevation is high).

References

[1] H. Zhao, *A fast sweeping method for Eikonal equations*, Mathematics of Computation, 74(250):603–627, 2005.

[2] Wikipedia contributors, *Eikonal equation*, Wikipedia, The Free Encyclopedia, https://en.wikipedia.org/wiki/Eikonal_equation.

[3] Harris, Charles R., et al. *Array programming with NumPy*. Nature 585, 357–362 (2020).